

E-Commerce & Retail Analytics

Database : Retail Analytics
(retail_sales.csv)

DATA QUALITY CHECKS

The screenshot displays the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the database structure, including the 'RetailAnalytics' database. The main window shows a SQL query titled 'SQLQuery.sql...eva D (84))' with the following content:

```
-- STEP 1 - DATA QUALITY CHECKS
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT order_id) AS unique_orders,
    COUNT(DISTINCT customer_id) AS unique_customers,
    COUNT(DISTINCT product_id) AS unique_products,
    MIN(order_date) AS earliest_date,
    MAX(order_date) AS latest_date,
    SUM(CASE WHEN gross_revenue < 0 THEN 1 ELSE 0 END) AS negative_revenue,
    SUM(CASE WHEN quantity <= 0 THEN 1 ELSE 0 END) AS zero_qty_rows,
    SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS null_customers
FROM retail_sales;
```

The query results are displayed in a table with the following columns: total_rows, unique_orders, unique_customers, unique_products, earliest_date, latest_date, negative_revenue, zero_qty_rows, and null_customers. The results show 500,000 total rows, 99,298 unique customers, and 5,000 unique products. There are no negative revenues, zero quantity rows, or null customer IDs.

	total_rows	unique_orders	unique_customers	unique_products	earliest_date	latest_date	negative_revenue	zero_qty_rows	null_customers
1	500000	500000	99298	5000	2020-01-01	2024-12-30	0	0	0

The status bar at the bottom indicates 'Query executed successfully.' and '1 rows'.

Overall Business KPIs

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'RetailAnalytics' database selected. The central pane shows a SQL query titled 'SQLQuery.sql...eva D (84)*'. The query calculates various business KPIs from the 'retail_sales' table, filtered by 'payment_status = 'Paid''. The right pane shows the results of the query, which are displayed in a table with 11 columns and 1 row.

```
101 |  
102 | -- Overall business KPIs  
103 | SELECT  
104 |     FORMAT(COUNT(order_id), 'N0') AS total_orders,  
105 |     FORMAT(COUNT(DISTINCT customer_id), 'N0') AS unique_customers,  
106 |     FORMAT(SUM(gross_revenue)/1e7, 'N2') + ' Cr' AS gross_revenue_cr,  
107 |     FORMAT(SUM(net_revenue)/1e7, 'N2') + ' Cr' AS net_revenue_cr,  
108 |     FORMAT(SUM(gross_profit)/1e7, 'N2') + ' Cr' AS gross_profit_cr,  
109 |     FORMAT(AVG(gross_margin_pct)*100, 'N1') + '%' AS avg_gross_margin,  
110 |     FORMAT(AVG(gross_revenue), 'N0') AS avg_order_value,  
111 |     FORMAT(SUM(return_value)/SUM(gross_revenue)*100, 'N1') + '%' AS return_rate,  
112 |     FORMAT(SUM(discount_amt)/SUM(gross_revenue+discount_amt)*100, 'N1') + '%' AS avg_discount_depth,  
113 |     FORMAT(AVG(CAST(rating AS FLOAT)), 'N2') AS avg_rating,  
114 |     FORMAT(AVG(CAST(delivery_days AS FLOAT)), 'N1') + ' days' AS avg_delivery_days  
115 | FROM retail_sales  
116 | WHERE payment_status = 'Paid';  
117 |  
118 |  
119 |  
120 |
```

100 % No issues found Ln: 101, Ch: 1 SPC LF UTF-8

	total_orders	unique_customers	gross_revenue_cr	net_revenue_cr	gross_profit_cr	avg_gross_margin	avg_order_value	return_rate	avg_discount_depth	avg_rating	avg_delivery_days
1	446,728	98,821	973.38 Cr	860.68 Cr	217.83 Cr	31.8%	21,789	11.6%	14.1%	4.02	4.0 days

Query executed successfully. LAPTOP-05JH0GRS\SQLEXPRESS ... LAPTOP-05JH0GRS\Jeeva ... RetailAnalytics 00:00:02 Row: 1, Col: 1 1 rows

Revenue from gross to net

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the Object Explorer with the following structure:

- Connect
- LAPTOP-O5JH0GRS\SQLEXPRESS (SQL Server 16.0.1180 - L)
- Databases
 - System Databases
 - Database Snapshots
 - Analytics
 - RetailAnalytics
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.retail_sales
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

The right pane shows a query window titled "SQLQuery.sql...eva D (84))" with the following SQL code:

```
-- Revenue waterfall from gross to net
SELECT
    ROUND(SUM(gross_revenue + discount_amt),0) AS mrp_revenue,
    ROUND(SUM(discount_amt),0) AS total_discounts,
    ROUND(SUM(gross_revenue),0) AS gross_revenue,
    ROUND(SUM(return_value),0) AS total_returns,
    ROUND(SUM(net_revenue),0) AS net_revenue,
    ROUND(SUM(cogs),0) AS total_cogs,
    ROUND(SUM(shipping_cost),0) AS total_shipping_cost,
    ROUND(SUM(gross_profit) - SUM(shipping_cost),0) AS contribution_margin
FROM retail_sales;
```

The status bar indicates "100 %", "No issues found", and "Ln: 115, Ch: 18 SPC LF UTF-8".

The Results pane shows the following data:

	mrp_revenue	total_discounts	gross_revenue	total_returns	net_revenue	total_cogs	total_shipping_cost	contribution_margin
1	126777191928.00	1790271596.00	10886920332.00	1263977180.00	9622943152.00	8449308556.00	51866321.00	2385745455.00

The status bar at the bottom indicates "Query executed successfully." and "LAPTOP-O5JH0GRS\SQLEXPRESS ... LAPTOP-O5JH0GRS\Jeeva ... RetailAnalytics 00:00:00 Row: 1, Col: 1 | 1 rows".

SALES TREND ANALYSIS

Monthly revenue trend with MoM growth

The screenshot displays a SQL IDE interface with a query editor and a results pane. The query is a complex SQL statement designed to analyze monthly revenue trends and calculate month-over-month (MoM) growth. It uses window functions like LAG and LEAD to compare current month revenue with previous months and the same month from the previous year.

```
-- SALES TREND ANALYSIS ( Monthly revenue trend with MoM growth )
WITH monthly_rev AS (
    SELECT
        order_year,
        order_month,
        DATEFROMPARTS(order_year, order_month, 1) AS month_date,
        SUM(net_revenue) AS net_revenue,
        COUNT(DISTINCT order_id) AS orders,
        COUNT(DISTINCT customer_id) AS customers,
        AVG(gross_revenue) AS avg_order_value
    FROM retail_sales
    GROUP BY order_year, order_month
)
SELECT
    order_year,
    order_month,
    ROUND(net_revenue, 0) AS net_revenue,
    orders,
    customers,
    ROUND(avg_order_value, 0) AS avg_order_value,
    ROUND(LAG(net_revenue) OVER (ORDER BY month_date), 0) AS prev_month_revenue,
    ROUND((net_revenue - LAG(net_revenue) OVER (ORDER BY month_date))
        / NULLIF(LAG(net_revenue) OVER (ORDER BY month_date), 0) * 100, 2) AS mom_growth_pct,
    ROUND(LAG(net_revenue, 12) OVER (ORDER BY month_date), 0) AS same_month_last_year,
    ROUND((net_revenue - LAG(net_revenue, 12) OVER (ORDER BY month_date))
        / NULLIF(LAG(net_revenue, 12) OVER (ORDER BY month_date), 0) * 100, 2) AS yoy_growth_pct
FROM monthly_rev
ORDER BY month_date;
```

The results pane shows the output of the query, displaying 10 rows of data. The columns include order_year, order_month, net_revenue, orders, customers, avg_order_value, prev_month_revenue, mom_growth_pct, same_month_last_year, and yoy_growth_pct.

	order_year	order_month	net_revenue	orders	customers	avg_order_value	prev_month_revenue	mom_growth_pct	same_month_last_year	yoy_growth_pct
29	2022	5	162785685.00	8439	8088	21668.000000	162753717.00	0.020000	162568533.00	0.130000
30	2022	6	144321418.00	8145	7814	20216.000000	162785685.00	-11.340000	160489434.00	-10.070000
31	2022	7	165417052.00	8574	8211	21864.000000	144321418.00	14.620000	163892481.00	0.930000
32	2022	8	161799317.00	8499	8120	21326.000000	165417052.00	-2.190000	159971761.00	1.140000
33	2022	9	158905294.00	8059	7712	22313.000000	161799317.00	-1.790000	157229243.00	1.070000
34	2022	10	155468022.00	8308	7984	21359.000000	158905294.00	-2.160000	161509933.00	-3.740000
35	2022	11	160409930.00	8325	7964	21759.000000	155468022.00	3.180000	158343683.00	1.300000
36	2022	12	157475265.00	8416	8067	21361.000000	160409930.00	-1.830000	164691907.00	-4.380000
37	2023	1	164445296.00	8549	8217	21744.000000	157475265.00	4.430000	173399705.00	-5.160000
38	2023	2	147740931.00	7750	7471	21724.000000	164445296.00	-10.160000	147637733.00	0.070000
39	2023	3	163780670.00	8475	8122	21745.000000	147740931.00	10.860000	163258587.00	0.320000
40	2023	4	163117206.00	8408	8071	21777.000000	163780670.00	-0.410000	162753717.00	0.220000

Quarterly performance with running total

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the following structure:

- Connect
- Databases
 - System Databases
 - Database Snapshots
 - Analytics
 - RetailAnalytics
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.retail_sales
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

The right pane shows the SQL Query Editor with the following query:

```
-- Quarterly performance with running total
SELECT
    order_year,
    order_quarter,
    ROUND(SUM(net_revenue), 0) AS quarterly_revenue,
    ROUND(SUM(gross_profit), 0) AS quarterly_profit,
    ROUND(AVG(gross_margin_pct)*100, 2) AS avg_margin_pct,
    COUNT(DISTINCT order_id) AS total_orders,
    ROUND(SUM(SUM(net_revenue)) OVER (
        PARTITION BY order_year
        ORDER BY order_quarter
        ROWS UNBOUNDED PRECEDING
    ), 0) AS ytd_revenue,
    ROUND(SUM(net_revenue)*100.0 / SUM(SUM(net_revenue)) OVER (
        PARTITION BY order_year, 2) AS pct_of_year_revenue
FROM retail_sales
GROUP BY order_year, order_quarter
ORDER BY order_year, order_quarter;
```

The query results are displayed in a table with the following columns: order_year, order_quarter, quarterly_revenue, quarterly_profit, avg_margin_pct, total_orders, ytd_revenue, and pct_of_year_revenue. The results show data for the years 2020 and 2021, grouped by quarter.

	order_year	order_quarter	quarterly_revenue	quarterly_profit	avg_margin_pct	total_orders	ytd_revenue	pct_of_year_revenue
1	2020	Q1	475123027.00	120416574.00	31.950000	24685	475123027.00	24.780000
2	2020	Q2	481284867.00	121498422.00	31.810000	24894	956407894.00	25.100000
3	2020	Q3	485102596.00	122940759.00	31.890000	25105	1441510490.00	25.300000
4	2020	Q4	475856473.00	120770330.00	31.770000	24939	1917366963.00	24.820000
5	2021	Q1	477385167.00	120092380.00	31.800000	24581	477385167.00	24.810000
6	2021	Q2	481311024.00	123158595.00	31.980000	25088	958696191.00	25.010000
7	2021	Q3	481093484.00	122153687.00	31.860000	25367	1439789675.00	25.000000
8	2021	Q4	484545523.00	123235012.00	32.060000	24964	1924335198.00	25.180000
9	2022	Q1	484296025.00	121730684.00	31.670000	24831	484296025.00	25.310000
10	2022	Q2	469860819.00	119460375.00	31.830000	24849	954156845.00	24.550000
11	2022	Q3	486121662.00	122477496.00	31.780000	25132	1440278507.00	25.400000

The status bar at the bottom indicates: Query executed successfully. LAPTOP-05JH0GRS\SQLEXPRESS... LAPTOP-05JH0GRS\Jeeva... RetailAnalytics 00:00:01 Row: 1, Col: 1 20 rows

Day-of-week sales pattern

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the Object Explorer with the 'RetailAnalytics' database selected. The central pane shows a SQL query titled 'SQLQuery.sql...eva D (84)*'. The query is a SELECT statement that calculates various metrics grouped by the day of the week. The right pane shows the results of the query, which is a table with 7 rows and 7 columns. The status bar at the bottom indicates that the query was executed successfully.

Object Explorer

- Connect
- LAPTOP-O5JH0GRS\SQLEXPRESS
 - Databases
 - System Databases
 - Database Snapshots
 - Analytics
 - RetailAnalytics
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.retail_sales
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

SQLQuery.sql...eva D (84)*

```
-- Day-of-week sales pattern
SELECT
    order_day_of_week,
    DATEPART(WEEKDAY, order_date) AS day_number,
    COUNT(order_id) AS total_orders,
    ROUND(SUM(net_revenue), 0) AS total_revenue,
    ROUND(AVG(gross_revenue), 0) AS avg_order_value,
    ROUND(AVG(CAST(rating AS FLOAT)), 2) AS avg_rating
FROM retail_sales
GROUP BY order_day_of_week, DATEPART(WEEKDAY, order_date)
ORDER BY day_number;
```

70 % No issues found Ln: 194, Ch: 21 SPC LF UTF-8

Results Messages

	order_day_of_week	day_number	total_orders	total_revenue	avg_order_value	avg_rating
1	Sunday	1	71290	1379987790.00	21922.000000	4.02
2	Monday	2	71344	1361097824.00	21642.000000	4.02
3	Tuesday	3	71428	1366408129.00	21597.000000	4.02
4	Wednesday	4	71383	1387057766.00	21886.000000	4.02
5	Thursday	5	71494	1371025215.00	21684.000000	4.02
6	Friday	6	71298	1368766464.00	21719.000000	4.02
7	Saturday	7	71763	1388599963.00	21965.000000	4.02

Query executed successfully. LAPTOP-O5JH0GRS\SQLEXPRESS ... LAPTOP-O5JH0GRS\Jeeva ... RetailAnalytics 00:00:01 Row: 1, Col: 1 7 rows

Seasonal sale event impact

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the Object Explorer with the following tree structure:

- Connect
- LAPTOP-05JH0GRS\SQLEXPRESS
 - Databases
 - System Databases
 - Database Snapshots
 - Analytics
 - RetailAnalytics
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.retail_sales
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

The main pane shows a query window titled "SQLQuery.sql...eva D (84))". The query is as follows:

```
-- Seasonal sale event impact
SELECT
    discount_event,
    COUNT(order_id) AS total_orders,
    ROUND(SUM(gross_revenue), 0) AS gross_revenue,
    ROUND(SUM(discount_amt), 0) AS total_discount_given,
    ROUND(AVG(discount_pct)*100, 2) AS avg_discount_pct,
    ROUND(SUM(gross_profit), 0) AS gross_profit,
    ROUND(AVG(gross_margin_pct)*100, 2) AS avg_margin_pct,
    ROUND(SUM(return_value)/NULLIF(SUM(gross_revenue), 0)*100, 2) AS return_rate_pct
FROM retail_sales
GROUP BY discount_event
ORDER BY gross_revenue DESC;
```

The status bar indicates "70 %", "No issues found", and "Ln: 208, Ch: 29 SPC LF UTF-8".

The Results pane shows the following data:

	discount_event	total_orders	gross_revenue	total_discount_given	avg_discount_pct	gross_profit	avg_margin_pct	return_rate_pct
1	None	225181	5702488146.00	0.00	0.000000	1277531280.00	31.850000	10.920000
2	Member Discount	49886	1139806956.00	126512808.00	10.000000	255157326.00	31.910000	10.840000
3	Diwali Sale	60165	1105159014.00	419096485.00	27.510000	246340388.00	31.820000	12.680000
4	Flash Sale	50012	1018022873.00	253937556.00	19.990000	228066621.00	31.800000	10.990000
5	End of Season	49815	812888475.00	438802644.00	34.980000	182878452.00	31.910000	14.410000
6	Clearance	40048	563230893.00	462408385.00	45.020000	125976581.00	31.840000	15.790000
7	New Year	24893	545323975.00	89513718.00	14.030000	121661128.00	31.680000	10.970000

The status bar at the bottom indicates "Query executed successfully." and "LAPTOP-05JH0GRS\SQLEXPRESS ... LAPTOP-05JH0GRS\jeewa ... RetailAnalytics 00:00:01 Row: 1, Col: 1 7 rows".

PRODUCT & CATEGORY ANALYSIS

Category summary

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the database structure for 'RetailAnalytics'. The main window displays a SQL query titled 'SQLQuery.sql...eva D (84)*'. The query is a SELECT statement that calculates various metrics for each category, including order count, units sold, revenue, profit, margin, discount, return rate, average order value, average rating, and ranks based on revenue and margin. The query is executed, and the results are displayed in a table at the bottom.

```
-- PRODUCT & CATEGORY ANALYSIS
-- Category P&L summary
SELECT
    category,
    COUNT(order_id) AS orders,
    SUM(quantity) AS units_sold,
    ROUND(SUM(gross_revenue)/1e6,2) AS revenue_mn,
    ROUND(SUM(gross_profit)/1e6,2) AS profit_mn,
    ROUND(AVG(gross_margin_pct)+100,2) AS avg_margin_pct,
    ROUND(SUM(discount_amt)/1e6,2) AS discount_mn,
    ROUND(SUM(return_value)/SUM(gross_revenue)+100,2) AS return_rate_pct,
    ROUND(AVG(gross_revenue),0) AS avg_order_value,
    ROUND(AVG(CAST(rating AS FLOAT)),2) AS avg_rating,
    RANK() OVER (ORDER BY SUM(gross_revenue) DESC) AS revenue_rank,
    RANK() OVER (ORDER BY AVG(gross_margin_pct) DESC) AS margin_rank
FROM retail_sales
GROUP BY category
ORDER BY revenue_rank;
```

	category	orders	units_sold	revenue_mn	profit_mn	avg_margin_pct	discount_mn	return_rate_pct	avg_order_value	avg_rating	revenue_rank	margin_rank
1	Electronics	89957	179915	6218.93	934.28	15.010000	1024.72	12.960000	69132.000000	3.96	1	7
2	Home & Living	60293	120600	1307.59	490.73	37.540000	213.89	9.160000	21687.000000	4.05	2	4
3	Appliances	24880	49582	1298.35	208.09	16.010000	213.06	9.110000	52184.000000	4.06	3	6
4	Fashion	109833	219606	773.72	386.74	50.000000	127.19	13.150000	7045.000000	3.96	4	2
5	Sports	40090	80185	523.27	203.99	39.010000	85.92	9.120000	13052.000000	4.07	5	3
6	Grocery	100107	501271	433.61	56.35	12.990000	71.19	9.280000	4332.000000	4.06	6	8
7	Beauty	49708	99324	220.27	121.28	55.050000	35.87	9.130000	4431.000000	4.06	7	1
8	Books & Media	25132	126111	111.18	36.15	32.500000	18.43	9.100000	4424.000000	4.07	8	5

Query executed successfully.

Subcategory & category share

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'RetailAnalytics' database selected. The central pane shows a SQL query titled 'SQLQuery.sql...eva D (84)*'. The query is a SELECT statement that calculates various metrics for categories and subcategories, including order counts, revenue, and margins. The results pane at the bottom shows a table with 11 rows and 8 columns: category, subcategory, orders, revenue, pct_of_category, pct_of_total, avg_margin_pct, and return_rate_pct. The status bar at the bottom indicates that the query was executed successfully.

```
228
229 -- Subcategory with category share
230 SELECT
231     category,
232     subcategory,
233     COUNT(order_id) AS orders,
234     ROUND(SUM(gross_revenue),0) AS revenue,
235     ROUND(SUM(gross_revenue)*100.0 / SUM(SUM(gross_revenue)) OVER (
236         PARTITION BY category),2) AS pct_of_category,
237     ROUND(SUM(gross_revenue)*100.0 / SUM(SUM(gross_revenue)) OVER (,2)
238         AS pct_of_total,
239     ROUND(AVG(gross_margin_pct)*100,2) AS avg_margin_pct,
240     ROUND(SUM(return_value)/SUM(gross_revenue)*100,2) AS return_rate_pct
241 FROM retail_sales
242 GROUP BY category, subcategory
243 ORDER BY category, revenue DESC;
```

	category	subcategory	orders	revenue	pct_of_category	pct_of_total	avg_margin_pct	return_rate_pct
1	Appliances	Refrigerators	5005	265033819.00	20.410000	2.430000	16.010000	9.100000
2	Appliances	Washing Machines	5019	263429044.00	20.290000	2.420000	16.050000	9.700000
3	Appliances	ACs	4971	261043377.00	20.110000	2.400000	16.000000	9.100000
4	Appliances	TVs	4975	257303515.00	19.820000	2.360000	15.970000	8.910000
5	Appliances	Kitchen Appliances	4910	251540441.00	19.370000	2.310000	16.040000	8.700000
6	Beauty	Makeup	9991	44882911.00	20.380000	0.410000	55.040000	10.000000
7	Beauty	Fragrances	10068	44661560.00	20.280000	0.410000	55.070000	9.120000
8	Beauty	Haircare	10028	44057931.00	20.000000	0.400000	54.950000	9.030000
9	Beauty	Personal Care	9867	43545276.00	19.770000	0.400000	55.160000	8.790000
10	Beauty	Skincare	9754	43117901.00	19.580000	0.400000	55.060000	8.650000
11	Books & Media	Books	5099	22987346.00	20.680000	0.210000	32.500000	9.460000

Query executed successfully.

Price discount v/s volume

The screenshot displays the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the database structure for 'LAPTOP-O5JH0GRS\SQLEXPRESS', including 'RetailAnalytics' and 'dbo.retail_sales'. The central pane shows a SQL query titled 'SQLQuery.sql...eva D (84))' with the following code:

```
-- Price discount vs volume
SELECT
category,
CASE
WHEN discount_pct = 0 THEN '0% (No Discount)'
WHEN discount_pct <= 0.10 THEN '1-10%'
WHEN discount_pct <= 0.20 THEN '11-20%'
WHEN discount_pct <= 0.30 THEN '21-30%'
WHEN discount_pct <= 0.50 THEN '31-50%'
ELSE '50%+'
END AS discount_band,
COUNT(order_id) AS orders,
SUM(quantity) AS units_sold,
ROUND(AVG(gross_revenue), 0) AS avg_order_value,
ROUND(AVG(gross_margin_pct)*100, 2) AS avg_margin_pct,
ROUND(SUM(return_value)/SUM(gross_revenue)*100, 2) AS return_rate_pct
FROM retail_sales
GROUP BY category,
CASE
WHEN discount_pct = 0 THEN '0% (No Discount)'
WHEN discount_pct <= 0.10 THEN '1-10%'
WHEN discount_pct <= 0.20 THEN '11-20%'
WHEN discount_pct <= 0.30 THEN '21-30%'
WHEN discount_pct <= 0.50 THEN '31-50%'
ELSE '50%+'
END
ORDER BY category, discount_band;
```

The query results are displayed in a table with 11 rows and 7 columns. The status bar at the bottom indicates 'Query executed successfully.' and '48 rows'.

	category	discount_band	orders	units_sold	avg_order_value	avg_margin_pct	return_rate_pct
1	Appliances	0% (No Discount)	11243	22480	60603.000000	16.030000	8.470000
2	Appliances	1-10%	1452	2899	55976.000000	15.900000	7.550000
3	Appliances	11-20%	4148	8181	52246.000000	16.040000	8.790000
4	Appliances	21-30%	3234	6454	44986.000000	16.020000	8.690000
5	Appliances	31-50%	4118	8187	37492.000000	15.970000	12.940000
6	Appliances	50%+	685	1381	27906.000000	16.090000	14.230000
7	Beauty	0% (No Discount)	22470	45015	5183.000000	55.120000	8.250000
8	Beauty	1-10%	2958	5964	4772.000000	55.060000	8.950000
9	Beauty	11-20%	8089	16095	4356.000000	55.020000	8.720000
10	Beauty	21-30%	6571	13057	3824.000000	54.990000	8.870000
11	Beauty	31-50%	8290	16547	3167.000000	54.950000	13.520000

Top 20 products by revenue

SQLQuery.sql...eva D (84)*

```
-- Top 20 products by revenue
SELECT TOP 20
  product_id,
  category,
  subcategory,
  brand_tier,
  COUNT(order_id) AS orders,
  SUM(quantity) AS units_sold,
  ROUND(AVG(unit_price), 2) AS avg_unit_price,
  ROUND(SUM(gross_revenue), 0) AS total_revenue,
  ROUND(SUM(gross_profit), 0) AS total_profit,
  ROUND(AVG(gross_margin_pct)*100, 2) AS avg_margin_pct,
  ROUND(AVG(CAST(rating AS FLOAT)), 2) AS avg_rating,
  ROUND(SUM(return_value)/SUM(gross_revenue)*100, 2) AS return_rate_pct
FROM retail_sales
GROUP BY product_id, category, subcategory, brand_tier
ORDER BY total_revenue DESC;
```

70% No issues found Ln: 307, Ch: 1 SPC LF UTF-8

Results Messages

	product_id	category	subcategory	brand_tier	orders	units_sold	avg_unit_price	total_revenue	total_profit	avg_margin_pct	avg_rating	return_rate_pct
1	PRD00857	Electronics	Laptops	Mid-Range	7	15	60049.180000	806177.00	103969.00	13.240000	4.29	13.700000
2	PRD03016	Electronics	Wearables	Mid-Range	5	13	63522.190000	802640.00	120794.00	14.720000	4.2	14.440000
3	PRD04437	Electronics	Tablets	Mid-Range	6	15	55547.340000	769997.00	124590.00	15.110000	3.5	3.090000
4	PRD01433	Electronics	Tablets	Premium	4	12	62795.950000	726725.00	100824.00	14.270000	4.5	0.000000
5	PRD04264	Electronics	Tablets	Mid-Range	6	14	56711.140000	722302.00	82968.00	11.640000	4.33	0.000000
6	PRD00025	Electronics	Laptops	Mid-Range	5	13	58229.820000	720786.00	132433.00	17.070000	3.8	14.830000
7	PRD01972	Electronics	Laptops	Mid-Range	7	15	49200.550000	706816.00	107859.00	13.880000	3.86	10.200000
8	PRD00940	Electronics	Tablets	Budget	6	14	45384.310000	648759.00	62895.00	11.240000	4	0.000000
9	PRD04997	Electronics	Smartphones	Mid-Range	7	16	48121.410000	645312.00	108198.00	16.550000	4.14	0.000000
10	PRD00746	Electronics	Smartphones	Private Label	5	12	52745.120000	641231.00	81862.00	13.840000	4.6	0.000000
11	PRD01958	Electronics	Smartphones	Mid-Range	5	11	64833.100000	639852.00	94930.00	15.550000	3.2	7.730000

Query executed successfully. LAPTOP-05JH0GRS\SQLEXPRESS ... LAPTOP-05JH0GRS\jeeva ... RetailAnalytics 00:00:04 Row: 1, Col: 1 20 rows

CUSTOMER ANALYTICS

Customer segment performance

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'RetailAnalytics' database selected. The central pane shows a SQL query titled 'SQLQuery.sql...eva D (84)*'. The query is a SELECT statement that calculates various performance metrics for different customer segments. The right pane shows the query results in a table format. The status bar at the bottom indicates that the query was executed successfully.

```
-- CUSTOMER ANALYTICS
-- Customer segment performance
SELECT
    customer_segment,
    COUNT(DISTINCT customer_id) AS unique_customers,
    COUNT(order_id) AS total_orders,
    ROUND(COUNT(order_id)*1.0/COUNT(DISTINCT customer_id),1) AS orders_per_customer,
    ROUND(SUM(net_revenue),0) AS total_revenue,
    ROUND(SUM(net_revenue)/COUNT(DISTINCT customer_id),0) AS revenue_per_customer,
    ROUND(AVG(gross_revenue),0) AS avg_order_value,
    ROUND(SUM(loyalty_points_earned),0) AS total_points_earned,
    ROUND(SUM(return_value)/SUM(gross_revenue)*100,2) AS return_rate_pct,
    ROUND(AVG(CAST(nps_score AS FLOAT)),1) AS avg_nps
FROM retail_sales
GROUP BY customer_segment
ORDER BY revenue_per_customer DESC;
```

	customer_segment	unique_customers	total_orders	orders_per_customer	total_revenue	revenue_per_customer	avg_order_value	total_points_earned	return_rate_pct	avg_nps
1	Regular	82761	175168	2.10000000000000	3369795404.00	40717.000000	21665.000000	67395994	11.210000	7.3
2	Occasional	63245	99846	1.60000000000000	1929697418.00	30511.000000	21774.000000	38593879	11.240000	7.3
3	Loyal	52833	75182	1.40000000000000	1463186401.00	27695.000000	21946.000000	29263807	11.320000	7.3
4	Premium	52525	74666	1.40000000000000	1444612103.00	27503.000000	21842.000000	28892281	11.420000	7.3
5	New	52866	75138	1.40000000000000	1415651825.00	26778.000000	21788.000000	28313058	13.530000	7.2

Query executed successfully.

Customer Lifetime Value (CLV) calculation

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'RetailAnalytics' database selected. The central pane shows a SQL query titled 'SQLQuery.sql...eva D (84)*'. The query calculates the Customer Lifetime Value (CLV) for each customer segment and loyalty tier. The results pane at the bottom shows the output of the query, which includes columns for customer segment, loyalty tier, and various metrics used in the CLV calculation.

```
-- Customer Lifetime Value (CLV)
WITH customer_orders AS (
    SELECT
        customer_id,
        customer_segment,
        loyalty_tier,
        COUNT(DISTINCT order_id) AS total_orders,
        ROUND(SUM(net_revenue), 0) AS total_spent,
        MIN(order_date) AS first_purchase,
        MAX(order_date) AS last_purchase,
        DATEDIFF(DAY, MIN(order_date), MAX(order_date)) AS customer_lifespan_days
    FROM retail_sales
    WHERE payment_status = 'Paid'
    GROUP BY customer_id, customer_segment, loyalty_tier
)
SELECT
    customer_segment,
    loyalty_tier,
    COUNT(customer_id) AS customers,
    ROUND(AVG(total_orders), 1) AS avg_orders,
    ROUND(AVG(total_spent), 0) AS avg_clv,
    ROUND(MAX(total_spent), 0) AS max_clv,
    ROUND(AVG(total_spent / NULLIF(total_orders, 0)), 0) AS avg_order_value,
    ROUND(AVG(customer_lifespan_days), 0) AS avg_lifespan_days,
    ROUND(AVG(total_spent / NULLIF(customer_lifespan_days / 30, 0, 0)), 0) AS avg_monthly_value
FROM customer_orders
GROUP BY customer_segment, loyalty_tier
ORDER BY avg_clv DESC;
```

	customer_segment	loyalty_tier	customers	avg_orders	avg_clv	max_clv	avg_order_value	avg_lifespan_days	avg_monthly_value
1	Regular	Bronze	46494	1	26065.000000	463276.00	19381.000000	189	7853.000000
2	Regular	Silver	37444	1	23989.000000	356819.00	19147.000000	142	8548.000000
3	Occasional	Bronze	29991	1	22808.000000	345659.00	19317.000000	108	7533.000000
4	Occasional	Silver	23668	1	22451.000000	364232.00	19695.000000	83	10719.000000
5	Regular	Gold	26980	1	22378.000000	426615.00	19212.000000	94	7493.000000
6	Loyal	Bronze	23734	1	22322.000000	344150.00	19513.000000	81	7585.000000
7	Premium	Bronze	23522	1	21824.000000	358910.00	19165.000000	82	7469.000000
8	Loyal	Silver	18257	1	21538.000000	328681.00	19400.000000	59	9134.000000
9	Premium	Silver	18203	1	21482.000000	451097.00	19379.000000	61	7424.000000
10	New	Bronze	23491	1	21376.000000	387911.00	18843.000000	79	9298.000000
11	Occasional	Gold	16236	1	21040.000000	368450.00	19228.000000	54	5993.000000

Query executed successfully.

LAPTOP-O5JH0GRS\SQLEXPRESS ... LAPTOP-O5JH0GRS\Jeeva ... RetailAnalytics 00:00:04 Row: 1, Col: 1 20 rows

CHANNEL & GEOGRAPHIC ANALYSIS

Channel performance comparison

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'RetailAnalytics' database selected. The central pane shows a SQL query titled 'SQLQuery.sql...eva D (84)*'. The query is a SELECT statement that calculates various performance metrics for different channels. The right pane shows the results of the query, which is a table with 11 columns and 5 rows of data.

```
342
343 -- CHANNEL & GEOGRAPHIC ANALYSIS
344 -- channel performance comparison
345 SELECT
346     channel,
347     COUNT(DISTINCT customer_id) AS unique_customers,
348     COUNT(order_id) AS total_orders,
349     ROUND(SUM(gross_revenue)/1e6,2) AS revenue_mn,
350     ROUND(AVG(gross_revenue),0) AS avg_order_value,
351     ROUND(AVG(gross_margin_pct)*100,2) AS avg_margin_pct,
352     ROUND(SUM(discount_amt)/SUM(gross_revenue+discount_amt)*100,2) AS discount_rate_pct,
353     ROUND(SUM(return_value)/SUM(gross_revenue)*100,2) AS return_rate_pct,
354     ROUND(AVG(CAST(delivery_days AS FLOAT)),1) AS avg_delivery_days,
355     ROUND(AVG(CAST(rating AS FLOAT)),2) AS avg_rating,
356     ROUND(SUM(gross_revenue)*100.0/SUM(SUM(gross_revenue)) OVER(),2) AS channel_revenue_share_pct
357
358 FROM retail_sales
359 GROUP BY channel
360 ORDER BY revenue_mn DESC;
```

100 % No issues found Ln: 360, Ch: 25 SPC LF UTF-8

	channel	unique_customers	total_orders	revenue_mn	avg_order_value	avg_margin_pct	discount_rate_pct	return_rate_pct	avg_delivery_days	avg_rating	channel_revenue_share_pct
1	Online	84960	189895	4132.33	21761.000000	31.840000	14.120000	13.570000	4	3.97	37.960000
2	In-Store	77635	150162	3271.89	21789.000000	31.840000	14.160000	10.090000	4	4.05	30.050000
3	Mobile App	67043	110572	2390.63	21621.000000	31.820000	14.050000	10.710000	4	4.05	21.960000
4	WhatsApp Commerce	25681	29661	660.68	22274.000000	31.910000	14.260000	10.470000	4	4.05	6.070000
5	Catalogue	17784	19710	431.39	21887.000000	31.850000	14.100000	11.040000	4	4.05	3.960000

Query executed successfully. LAPTOP-O5JH0GRS\SQLEXPRESS ... LAPTOP-O5JH0GRS\Jeeva ... RetailAnalytics 00:00:01 Row: 1, Col: 1 5 rows

State-level revenue and growth

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'RetailAnalytics' database selected. The main pane shows a SQL query titled 'SQLQuery.sql...eva D (84)*'. The query is as follows:

```
-- State-level revenue and growth
WITH state_rev AS (
    SELECT
        state,
        order_year,
        SUM(net_revenue) AS revenue
    FROM retail_sales
    GROUP BY state, order_year
)
SELECT
    state,
    ROUND(SUM(CASE WHEN order_year=2022 THEN revenue ELSE 0 END),0) AS rev_2022,
    ROUND(SUM(CASE WHEN order_year=2023 THEN revenue ELSE 0 END),0) AS rev_2023,
    ROUND(SUM(CASE WHEN order_year=2024 THEN revenue ELSE 0 END),0) AS rev_2024,
    ROUND((SUM(CASE WHEN order_year=2024 THEN revenue ELSE 0 END)
        - SUM(CASE WHEN order_year=2023 THEN revenue ELSE 0 END))
        / NULLIF(SUM(CASE WHEN order_year=2023 THEN revenue ELSE 0 END),0)*100,2)
        AS yoy_growth_pct_2024,
    RANK() OVER (ORDER BY SUM(CASE WHEN order_year=2024 THEN revenue ELSE 0 END) DESC)
        AS rev_rank_2024
FROM state_rev
GROUP BY state
ORDER BY rev_rank_2024;
```

The bottom pane shows the results of the query, displaying a table with 6 columns: state, rev_2022, rev_2023, rev_2024, yoy_growth_pct_2024, and rev_rank_2024. The results are sorted by rev_rank_2024 in descending order.

	state	rev_2022	rev_2023	rev_2024	yoy_growth_pct_2024	rev_rank_2024
1	Andhra Pradesh	129269987.00	129233009.00	133806680.00	3.540000	1
2	Kerala	129804063.00	132304009.00	132926743.00	0.470000	2
3	Telangana	126919971.00	125391707.00	131428595.00	4.810000	3
4	Maharashtra	128849271.00	131581878.00	130737353.00	-0.640000	4
5	Karnataka	127518382.00	132513829.00	130247204.00	-1.710000	5
6	West Bengal	128413029.00	131261282.00	130017323.00	-0.950000	6
7	Rajasthan	124741411.00	127578764.00	129266098.00	1.320000	7
8	Haryana	126809751.00	127871298.00	128496256.00	0.490000	8
9	Gujarat	128833944.00	125688890.00	127690545.00	1.590000	9
10	Delhi	130823388.00	129026052.00	127674780.00	-1.050000	10
11	Punjab	128548206.00	128925536.00	126839843.00	-1.620000	11

The status bar at the bottom indicates 'Query executed successfully.' and 'Row: 1, Col: 1 | 15 rows'.

City tier revenue

File Edit View Query Git Project Tools Extensions Window Help Search

RetailAnalytics Execute

Object Explorer

- Connect
- LAPTOP-O5JH0GRS\SQLEXPRESS
 - Databases
 - System Databases
 - Database Snapshots
 - Analytics
 - RetailAnalytics
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.retail_sales
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

SQLQuery.sql...eva D (84))*

```
--City tier revenue
SELECT
    city_tier,
    channel,
    COUNT(order_id) AS orders,
    ROUND(SUM(gross_revenue)/1e6,2) AS revenue_mn,
    ROUND(AVG(gross_revenue),0) AS avg_order_value,
    ROUND(SUM(return_value)/SUM(gross_revenue)*100,2) AS return_rate_pct
FROM retail_sales
GROUP BY city_tier, channel
ORDER BY city_tier, revenue_mn DESC;
```

100 % No issues found Ln: 396, Ch: 17 SPC LF UTF-8

Results Messages

	city_tier	channel	orders	revenue_mn	avg_order_value	return_rate_pct
1	Tier 1	Online	66725	1463.56	21934.000000	13.300000
2	Tier 1	In-Store	52432	1148.76	21910.000000	9.930000
3	Tier 1	Mobile App	38802	840.3	21656.000000	10.670000
4	Tier 1	WhatsApp Commerce	10257	223.99	21838.000000	9.870000
5	Tier 1	Catalogue	6862	151.35	22057.000000	11.180000
6	Tier 2	Online	75829	1644.94	21693.000000	13.800000
7	Tier 2	In-Store	60117	1302	21658.000000	10.270000
8	Tier 2	Mobile App	44427	968.89	21809.000000	10.510000
9	Tier 2	WhatsApp Commerce	11970	269.24	22493.000000	10.530000
10	Tier 2	Catalogue	7860	169.95	21623.000000	11.330000
11	Tier 3	Online	47341	1023.83	21627.000000	13.610000

Query executed successfully. LAPTOP-O5JH0GRS\SQLEXPRESS ... LAPTOP-O5JH0GRS\Jeeva ... RetailAnalytics 00:00:00 Row: 1, Col: 1 15 rows

RETURN RATE & QUALITY

Return rate by category and channel

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'RetailAnalytics' database selected. The central pane shows a SQL query titled 'SQLQuery.sql...eva D (84)*'. The query is a SELECT statement that calculates the return rate and quality by category and channel. The right pane shows the results of the query, which is a table with 7 columns: category, channel, total_orders, returned_orders, return_rate_pct, total_return_value, and return_value_pct. The results are sorted by return_rate_pct in descending order.

```
397 /
398 -- RETURN RATE & QUALITY ANALYSIS
399 -- Return rate by category and channel
400 SELECT
401     category,
402     channel,
403     COUNT(order_id) AS total_orders,
404     SUM(return_flag) AS returned_orders,
405     ROUND(SUM(return_flag)*100.0/COUNT(order_id), 2) AS return_rate_pct,
406     ROUND(SUM(return_value), 0) AS total_return_value,
407     ROUND(SUM(return_value)/SUM(gross_revenue)*100, 2) AS return_value_pct
408 FROM retail_sales
409 GROUP BY category, channel
410 ORDER BY return_rate_pct DESC;
411
412
413
414
415
416
417
418
419
420
```

100 % No issues found Ln: 420, Ch: 1 SPC LF UTF-8

	category	channel	total_orders	returned_orders	return_rate_pct	total_return_value	return_value_pct
1	Fashion	Online	41453	6393	15.4200000000000	44252528.00	15.160000
2	Electronics	Online	34236	5184	15.1400000000000	354197248.00	15.030000
3	Electronics	Catalogue	3536	462	13.0700000000000	31572363.00	12.810000
4	Fashion	Catalogue	4350	538	12.3700000000000	3887470.00	12.610000
5	Fashion	Mobile App	24459	3024	12.3600000000000	20678465.00	12.100000
6	Electronics	Mobile App	19652	2427	12.3500000000000	165809754.00	12.230000
7	Fashion	In-Store	33140	4065	12.2700000000000	27477186.00	11.720000
8	Electronics	WhatsApp Commerce	5416	654	12.0800000000000	45212483.00	11.790000
9	Fashion	WhatsApp Commerce	6431	777	12.0800000000000	5437609.00	11.930000
10	Electronics	In-Store	27117	3263	12.0300000000000	209308801.00	11.160000
11	Books & Media	Online	9496	1087	11.4500000000000	4714667.00	11.300000

Query executed successfully. LAPTOP-O5JH0GRS\SQLEXPRESS ... LAPTOP-O5JH0GRS\Jeeva ... RetailAnalytics 00:00:00 Row: 1, Col: 1 40 rows

Return reason with financial impact

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the Object Explorer with the following structure:

- Connect
- LAPTOP-05JH0GRS\SQLEXPRESS
 - Databases
 - System Databases
 - Database Snapshots
 - Analytics
 - RetailAnalytics
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.retail_sales
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

The right pane shows the SQL Query Editor with the following query:

```
-- Return reason with financial impact
SELECT
    return_reason,
    COUNT(order_id) AS return_count,
    ROUND(SUM(return_value),0) AS total_return_value,
    ROUND(AVG(return_value),0) AS avg_return_value,
    ROUND(SUM(return_value)*100.0/SUM(SUM(return_value)) OVER(),2) AS pct_of_total_returns,
    ROUND(AVG(CAST(rating AS FLOAT)),2) AS avg_rating,
    (SELECT TOP 1 category FROM retail_sales r2
     WHERE r2.return_reason = r1.return_reason
     GROUP BY category ORDER BY COUNT(*) DESC) AS top_category
FROM retail_sales r1
WHERE return_flag = 1
GROUP BY return_reason
ORDER BY total_return_value DESC;
```

The status bar indicates "No issues found" and "Ln: 428, Ch: 34 SPC LF UTF-8".

The Results pane shows the following data:

	return_reason	return_count	total_return_value	avg_return_value	pct_of_total_returns	avg_rating	top_category
1	Defective	15479	359875748.00	23249.000000	28.470000	1.74	Fashion
2	Wrong Size	12081	274804756.00	22747.000000	21.740000	1.75	Fashion
3	Not as Described	10913	250895540.00	22991.000000	19.850000	1.76	Fashion
4	Changed Mind	8288	192421431.00	23217.000000	15.220000	1.76	Fashion
5	Damaged in Transit	5630	127160664.00	22586.000000	10.060000	1.74	Fashion
6	Duplicate Order	2656	58819041.00	22146.000000	4.650000	1.74	Fashion

The status bar at the bottom indicates "Query executed successfully." and "LAPTOP-05JH0GRS\SQLEXPRESS ... LAPTOP-05JH0GRS\Jeeva ... RetailAnalytics 00:00:00 Row: 1, Col: 1 6 rows".

PAYMENT & LOYALTY

Payment method breakdown

The screenshot displays the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the database structure for 'RetailAnalytics'. The central pane shows a SQL query titled 'SQLQuery.sql...eva D (84))' with the following content:

```
-- PAYMENT & LOYALTY ANALYTICS
-- Payment method breakdown
SELECT
    payment_method,
    payment_status,
    COUNT(order_id) AS orders,
    ROUND(SUM(gross_revenue), 0) AS revenue,
    ROUND(AVG(gross_revenue), 0) AS avg_order_value,
    ROUND(SUM(return_value)/SUM(gross_revenue)*100, 2) AS return_rate_pct
FROM retail_sales
GROUP BY payment_method, payment_status
ORDER BY payment_method, orders DESC;
```

The bottom pane shows the results of the query, displaying a table with 7 columns: payment_method, payment_status, orders, revenue, avg_order_value, and return_rate_pct. The results are sorted by payment_method and then by orders in descending order.

	payment_method	payment_status	orders	revenue	avg_order_value	return_rate_pct
1	BNPL	Paid	22508	481399495.00	21388.000000	11.290000
2	BNPL	Pending	992	20992497.00	21162.000000	13.020000
3	BNPL	Failed	744	15884408.00	21350.000000	13.350000
4	BNPL	Refunded	705	15432923.00	21891.000000	13.160000
5	COD	Paid	59249	1292707896.00	21818.000000	11.150000
6	COD	Pending	6886	146995785.00	21347.000000	11.970000
7	COD	Failed	3506	77909683.00	22222.000000	10.710000
8	Credit Card	Paid	81221	1767247136.00	21759.000000	11.540000
9	Credit Card	Pending	3646	83849139.00	22998.000000	12.110000
10	Credit Card	Failed	2749	59878022.00	21782.000000	11.020000
11	Credit Card	Refunded	2669	59644671.00	22347.000000	11.140000

The status bar at the bottom indicates 'Query executed successfully' and shows the current file path: 'LAPTOP-OSJH0GRS\SQL EXPRESS - LAPTOP-OSJH0GRS\leaves - RetailAnalytics - 00:00:00 - Row 1 Col 1 - 31 rows'.

Market basket which categories are bought together

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'LAPTOP-05JH0GRS\SQLEXPRESS', including 'RetailAnalytics' and 'dbo.retail_sales'. The main query editor displays a T-SQL query to find categories bought together, calculating co-occurrence and support percentage. The query is as follows:

```
-- Market basket which categories are bought together
SELECT
    a.category AS category_1,
    b.category AS category_2,
    COUNT(*) AS co_occurrence,
    ROUND(COUNT(*)*100.0 / (
        SELECT COUNT(DISTINCT customer_id) FROM retail_sales
    ),4) AS support_pct
FROM retail_sales a
JOIN retail_sales b
    ON a.customer_id = b.customer_id
   AND a.category < b.category
   AND a.order_id <= b.order_id
GROUP BY a.category, b.category
ORDER BY co_occurrence DESC;
```

The Results pane at the bottom shows the output of the query, displaying 11 rows of category pairs, their co-occurrence counts, and support percentages. The status bar indicates the query was executed successfully.

	category_1	category_2	co_occurrence	support_pct
1	Fashion	Grocery	110181	110.959900000000
2	Electronics	Fashion	98970	99.669700000000
3	Electronics	Grocery	90137	90.774200000000
4	Fashion	Home & Living	66607	67.077900000000
5	Grocery	Home & Living	60264	60.690000000000
6	Beauty	Fashion	54880	55.268000000000
7	Electronics	Home & Living	54297	54.680900000000
8	Beauty	Grocery	49745	50.096700000000
9	Beauty	Electronics	44823	45.139900000000
10	Fashion	Sports	44108	44.419800000000
11	Grocery	Sports	40292	40.576800000000

Query executed successfully. | LAPTOP-05JH0GRS\SQLEXPRESS ... | LAPTOP-05JH0GRS\Jeeva ... | RetailAnalytics | 00:00:02 | Row: 1, Col: 1 | 28 rows

PIVOT: Revenue by channel across years

The screenshot displays the SQL Server Enterprise Manager interface. The left-hand pane shows the 'Object Explorer' with a tree view of the database structure, including 'RetailAnalytics' and 'dbo.retail_sales'. The central pane contains a T-SQL query titled 'SQLQuery.sql...eva D (84)*'. The query is a PIVOT statement designed to calculate revenue by channel across five years (2020-2024). The right-hand pane shows the results of the query, which is a table with 6 columns: 'channel', '2020', '2021', '2022', '2023', and '2024'. The results are ordered by the year 2024 in descending order. The status bar at the bottom indicates that the query was executed successfully.

```
458
459 -- PIVOT Revenue by channel across years
460 SELECT *
461 FROM (
462     SELECT channel, order_year,
463            ROUND(SUM(gross_revenue)/1e6,2) AS revenue_mn
464     FROM retail_sales
465     GROUP BY channel, order_year
466 ) src
467 PIVOT (
468     SUM(revenue_mn)
469     FOR order_year IN ([2020],[2021],[2022],[2023],[2024])
470 ) pvt
471 ORDER BY [2024] DESC;
```

channel	2020	2021	2022	2023	2024
1 Online	831.02	824.19	815	827.3	834.83
2 In-Store	644.73	656.16	650.83	663.94	656.22
3 Mobile App	473.55	478.56	476.86	477.49	484.16
4 WhatsApp Commerce	131.71	131.63	134.63	128.7	134.01
5 Catalogue	86.17	85.54	89.49	86.64	83.54

Query executed successfully.

Running total and market share by category per month

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the Object Explorer with the following structure:

- Connect
- LAPTOP-05JH0GRS\SQLEXPRESS
 - Databases
 - System Databases
 - Database Snapshots
 - Analytics
 - RetailAnalytics
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.retail_sales
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

The main pane shows the query editor with the following SQL code:

```
-- Running total and market share by category per month
SELECT
    order_year,
    order_month,
    category,
    ROUND(SUM(gross_revenue),0) AS monthly_revenue,
    ROUND(SUM(gross_revenue)*100.0/SUM(SUM(gross_revenue)) OVER (
        PARTITION BY order_year, order_month, 2) AS market_share_pct,
    ROUND(SUM(SUM(gross_revenue)) OVER (
        PARTITION BY order_year, category
        ORDER BY order_month
        ROWS UNBOUNDED PRECEDING),0) AS ytd_revenue
FROM retail_sales
GROUP BY order_year, order_month, category
ORDER BY order_year, order_month, monthly_revenue DESC;
```

The status bar at the bottom indicates "Query executed successfully." and "No issues found". The bottom right corner shows the connection string "LAPTOP-05JH0GRS\SQLEXPRESS ..." and the current row information "Row: 1, Col: 1 | 480 rows".

	order_year	order_month	category	monthly_revenue	market_share_pct	ytd_revenue
1	2020	1	Electronics	108689033.00	57.840000	108689033.00
2	2020	1	Appliances	21477591.00	11.430000	21477591.00
3	2020	1	Home & Living	21252822.00	11.310000	21252822.00
4	2020	1	Fashion	13987565.00	7.440000	13987565.00
5	2020	1	Sports	9757342.00	5.190000	9757342.00
6	2020	1	Grocery	7205799.00	3.830000	7205799.00
7	2020	1	Beauty	3714835.00	1.980000	3714835.00
8	2020	1	Books & Media	1816995.00	0.970000	1816995.00
9	2020	2	Electronics	90761788.00	55.430000	199450821.00
10	2020	2	Appliances	20245088.00	12.360000	41722679.00
11	2020	2	Home & Living	19809079.00	12.100000	41061901.00

Top N products per category (ranking per group)

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'RetailAnalytics' database selected. The central pane shows a SQL query titled 'SQLQuery.sql...eva D (84))' with the following code:

```
-- Top N products per category (ranking per group)
WITH ranked_products AS (
    SELECT
        category,
        product_id,
        subcategory,
        ROUND(SUM(gross_revenue),0) AS revenue,
        COUNT(order_id) AS orders,
        ROUND(AVG(gross_margin_pct)*100,2) AS margin_pct,
        DENSE_RANK() OVER (
            PARTITION BY category
            ORDER BY SUM(gross_revenue) DESC
        ) AS rank_in_category
    FROM retail_sales
    GROUP BY category, product_id, subcategory
)
SELECT * FROM ranked_products
WHERE rank_in_category <= 3
ORDER BY category, rank_in_category;
```

The bottom pane shows the results of the query, displaying a table with 11 rows and 7 columns. The status bar at the bottom indicates 'Query executed successfully.' and 'Row: 1, Col: 1 | 24 rows'.

	category	product_id	subcategory	revenue	orders	margin_pct	rank_in_category
1	Appliances	PRD04845	ACs	629846.00	6	14.930000	1
2	Appliances	PRD02551	Refrigerators	488757.00	4	15.610000	2
3	Appliances	PRD00981	Refrigerators	484556.00	7	18.070000	3
4	Beauty	PRD00454	Skincare	70144.00	8	57.660000	1
5	Beauty	PRD00297	Skincare	60839.00	9	58.210000	2
6	Beauty	PRD00994	Haircare	59664.00	10	52.210000	3
7	Books & Media	PRD00641	E-Learning	51330.00	7	34.250000	1
8	Books & Media	PRD02298	Music	46277.00	5	34.660000	2
9	Books & Media	PRD04767	Books	45325.00	5	30.070000	3
10	Electronics	PRD00002	Smartphones	1200338.00	9	14.430000	1
11	Electronics	PRD01958	Smartphones	1170485.00	10	15.520000	2